IoT 機器における不揮発性メモリへの書込み成立条件に基づくマルウェア永続化の脅威分析

西岡 謙介[1,2,a)]　竹久 達也[1,3,2]　奥川 莞多[1,2]　毛利 公一[1,2]　久保 正樹[1]　笠間 貴弘[1]

概要：IoT 機器へのマルウェア感染が報告される中，マルウェアが自身を不揮発性メモリへ保存し，自動実行設定を改変することで，再起動後も感染状態を維持する永続化が問題となっている．永続化には不揮発性メモリへの書込みが必要であり，書込み可否には，実行主体に対するアクセス制御に加え，ユーザ空間から不揮発性メモリに至る各層の構成や保護状態などが関係する．既存研究では，実機上でマルウェアを動的解析し，永続化の可能性を分析しているが，書込み可否を規定する各層の要因とその組合せは体系的に整理されていない．そこで本研究では，Linux ベースの IoT 機器における不揮発性メモリへの書込み経路を階層ごとに整理し，各層の要因とその組合せに基づく書込み成立条件を明らかにする．さらに，書込み成立判定に必要な情報を取得するツールを開発し，実機における書込み可否に基づいてマルウェア永続化の成立可能性を評価するとともに，永続化対策における設計上の課題を考察する．

キーワード：永続化，不揮発性メモリ，IoT マルウェア

Threat Analysis of Malware Persistence in IoT Devices Based on Conditions Enabling Writes to Non-Volatile Memory

Kensuke Nishioka[1,2,a)]　Tatsuya Takehisa[1,3,2]　Kanta Okugawa[1,2]　Koichi Mouri[1,2]
Masaki Kubo[1]　Takahiro Kasama[1]

Abstract: Amid reports of malware infections affecting IoT devices, malware persistence has become a major concern. Malware can establish persistence by storing itself in non-volatile memory and modifying autostart configurations to maintain the infection after a reboot. Establishing persistence in this way requires writing to non-volatile memory. Whether such writes can succeed depends not only on the access controls applied to the subject performing the write but also on the configuration and protection state at each layer along the path from user space to non-volatile memory. Prior studies have conducted dynamic malware analysis on physical devices to assess whether persistence can be established. However, the factors at each layer that determine whether such writes can succeed, as well as the combinations of these factors, have not been systematically organized. This study therefore categorizes the write paths from user space to non-volatile memory in Linux-based IoT devices layer by layer, and clarifies the conditions for successful writes based on the relevant factors and their combinations. We also develop a tool that collects the information needed to determine whether such writes can succeed on physical devices. Based on these results, we evaluate the feasibility of malware persistence and discuss design challenges for countermeasures against it.

Keywords: Persistence, Non-Volatile Memory, IoT Malware

1. はじめに

IoT マルウェア Mirai の登場以降，脆弱な IoT 機器がマルウェアに感染し，サイバー攻撃の踏み台や DDoS 攻撃な

[1] 国立研究開発法人情報通信研究機構，National Institute of Information and Communications Technology
[2] 立命館大学，Ritsumeikan University
[3] 株式会社ニッシン，Nissin Inc.
[a)] knishioka@nict.go.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

どに悪用される事例が報告されている [1]．Mirai の亜種を含む多くの IoT マルウェアは，自身の感染状態を維持・長期化するために，競合マルウェアの停止やプロセスの隠蔽などの機能を持つことが知られている [2]．このような感染長期化の脅威の 1 つとして，機器の再起動後もマルウェアが活動状態を維持する永続化がある．永続化したマルウェアは，単純なプロセス停止や機器の再起動では除去が難しい [3], [4], [5], [6]．実際に，永続化を試みるマルウェアとして，VPNFilter [7] や SYNful Knock [8] が報告されている．

このように，マルウェアが再起動後も活動を維持する際には，不揮発性メモリに対して永続化に必要なコードや設定を書き込む．具体的には，不揮発性メモリにマルウェアのバイナリを直接保存する場合や，マルウェアをダウンロードするスクリプトを保存する場合，再起動後にこれらを実行する設定を書き込む場合がある．一方，マルウェアがこれらの書込み処理を備えていても，侵入先の IoT 機器上で実際に書込みが成立するとは限らない．Linux では，実行主体の権限に加えて，ファイルシステムの構成や書込み保護の状態も書込みの成否に影響する．

既存研究では，マルウェアが利用する自動実行機構や複数の永続化手法について分析されている [3], [4], [5]．しかし，マルウェアが発行する書込み要求が対象機器上で成立する条件については，ユーザ空間から不揮発性メモリまでの書込み経路上に存在する各層の書込み条件とその組合せという観点から，十分に調査されていない．

そこで本研究では，Linux ベースの IoT 機器を対象とし，ユーザ空間から不揮発性メモリに至る書込み経路において書込みの成否を決定する各層の条件とその関係を明らかにする．また，これらの条件に関する情報を実機から取得し，新規ファイルの作成および自動実行設定の変更可否を判定するツールを開発し，検証用機器および 2 機種の SOHO ルータに適用し，対象機器におけるマルウェア永続化の成立可能性を評価する．

本研究の貢献は次の通りである．

- **不揮発性メモリへの書込み成立条件の体系化**：Linux における通常ファイルへの書込みを対象として，ユーザ空間から不揮発性メモリに至る経路を階層化し，各層で書込みの成否を決定する要因とその関係を整理した．
- **診断ツールの開発**：書込み成立条件について各層の設定および状態を実機から取得し，新規ファイルの作成と自動実行設定の変更可否を判定するツールを開発した．
- **実機における永続化可能性の調査**：開発したツールを検証用機器および 2 機種の SOHO ルータに適用し，診断結果と実際の書込み結果を比較するとともに，機器の構成によってマルウェア永続化の成立可能性が異なることを示した．

2. 背景と関連研究

2.1 Linux におけるマルウェアの永続化とその脅威

マルウェアの永続化には，大きく分けて 2 つの手法が考えられる．1 つは，マルウェア本体を不揮発性メモリ上に保存し，再起動後に当該マルウェアを実行するための設定を保持する方法である．もう 1 つは，再起動後にマルウェアを再取得し，実行するスクリプトまたはコマンドを自動実行設定に登録し，感染状態を維持する方法である．いずれの場合も，マルウェア本体，スクリプト，または自動実行設定を，再起動後も保持される領域へ書き込む必要がある．

本研究では，脅威モデルとしてこれらを実現する手段の 1 つであるファイルシステムを介したマルウェアの永続化に着目する．この脅威モデルは，初期侵入後に IoT 機器上で動作するマルウェアが，自身のバイナリまたはマルウェアを再取得するスクリプトを保存する操作と，再起動後に当該プログラムを実行させるための自動実行設定を新規作成または改変する操作を対象とする．

脅威モデルおよびファイルシステムを介した不揮発性メモリへの書込み経路を図 1 に示す．マルウェアによるファイルの新規作成または改変は，ユーザ空間から発行されるシステムコールを起点として，VFS を介して対象のファイルシステムへ渡される．対象が read-only ファイルシステムである場合，これらの書込みは成立しない．一方，書込み可能であり，書込み内容が不揮発性メモリに保持されるファイルシステムでは，下位のデバイスサブシステムを経由して書込み内容が不揮発性メモリへ反映され，再起動後も保持され得る．

しかし，侵入後のマルウェアが書込み要求を発行できたとしても，マルウェア本体の保存や自動実行設定の改変が常に成立するとは限らない．その成否は，実行主体の権限，対象ファイルおよびディレクトリに対するアクセス制御，ファイルシステムの書込み可否，マウントポイントの構成，および下位のデバイスサブシステム等の書込み保護状態に依存する．そのため，このような脅威を分析するには，ファイル作成および改変が再起動後まで保持される条件を分析する必要がある．

2.2 関連研究

IoT マルウェアの永続化については，実マルウェアの動的解析や実機を用いた実証を通じて，永続化に利用されるファイルや自動実行機構が調査されている [5], [9]．原らの研究 [5] では，起動スクリプト等を改変する感染持続型 IoT マルウェアの攻撃モデルを構築し，実機を用いた検証を行っている．この永続化の検証では，永続化の成否がファイルシステムの性質に依存することが示されている．一方，検証結果はマルウェアが備える機能に依存しており，書込みが不成立となった要因は階層的に分析されていない．

© 2026 Information Processing Society of Japan

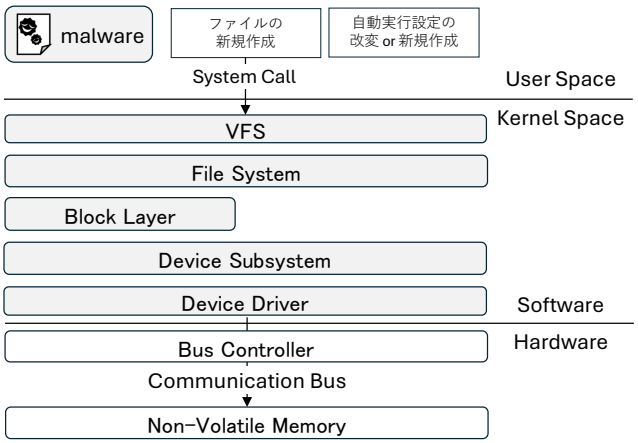


図 1 脅威モデルおよびファイルシステムを介した不揮発性メモリへの書込み経路

また，Alrawi ら [4] の研究では，init や rc，cron などの自動実行設定へのマルウェアの動作が報告されている．これらの研究は実際の IoT マルウェアがどのような設定や機能を永続化に利用するかを明らかにしている一方で，観測された書込み要求が対象の機器上で成立するための条件については分析が不十分である．Brierley らの研究 [3] では，Linux ベースの IoT 機器を対象として，initrd や initramfs の変更，ファームウェア更新機能の悪用，MTD（Memory Technology Device）の書込み保護を変更する方法などの複数の永続化手法について実験を行っている．この研究は，ファイルシステムより下位の機構を含めた永続化手法が実現可能であることを示している一方で，そのほかの制御機構の組合せを考慮した通常ファイルへの書込み成立条件として体系化されていない．

以上の既存研究では，永続化に利用されるファイルや自動実行機構の設定，個別の攻撃手法について分析が行われている．しかしながら，ユーザ空間の書込み要求に対して，不揮発性メモリに至る各層のアクセス制御や書込み保護状態についての整理が不十分であり，マルウェアの書込み成立条件は明らかになっていない．

3. Linux における不揮発性メモリへの書込み成立条件の調査

本章では，2.1 節で説明した脅威モデルで永続化を図るマルウェアによるファイルシステムを介した不揮発性メモリへの書込み成立条件について説明する．

3.1 調査方針

Linux システムにおいて，ユーザ空間から不揮発性メモリへのアクセスは最終的にシステムコールに集約される．したがって，Linux における通常ファイルへの書込みでは，ユーザ空間のプロセスが open または openat で対象ファイルを開き，取得したファイルディスクリプタに対して write 等を実行するといった処理がなされる．変更されたデータはファイルシステムによって処理された後，writeback 等を介して不揮発性メモリへ反映される．

また，本研究では実行主体の権限を固定せず，UID，GID，補助グループ，Linux Capability 等を書込み成立条件の一部として扱う．一方，/dev/mtdX 等のデバイスファイルを直接操作する raw access は通常ファイルとは異なる書込み経路を持つため，主要な分析対象とはしない．ただし，通常ファイルへの書込み経路の下位に位置する block device や MTD subsystem の保護状態は書込み成否に影響するため，本研究の分析対象に含める．

3.2 ファイル操作に対するアクセス制御と属性による制約

open および openat では，指定されたパスを基に対象となる mount，dentry，inode 等が特定され，書込み可能なファイルディスクリプタを生成できるかが判定される．この過程では，実行主体のアクセス権限だけでなく，ファイルおよびファイルシステムの状態が書込み可否に影響する．

新規ファイル作成では，作成対象の inode がまだ存在しないため，親ディレクトリに対する検索権限 x と書込み権限 w が必要となる．一方，既存ファイルを書き換える場合には，対象パスを探索するための親ディレクトリの検索権限 x と，対象ファイル自身に対する書込み権限 w が必要となる．これらの DAC による判定には mode bits に加えて POSIX ACL が影響し，プロセスが特定の Linux Capability を持つ場合には，一部の DAC 判定を回避できる場合がある．

さらに，LSM を用いて実装された強制アクセス制御や Seccomp によるシステムコール制限も書込み処理を阻害し得る．また，対象の inode または親ディレクトリに immutable や append-only 等の属性が設定されている場合には，DAC 上で操作が許可されていても，要求する操作内容に応じて変更が制限される．

3.3 ファイルシステムおよびマウントによる制約

アクセス制御上の書込みが許可されている場合でも，対象ファイルが存在するファイルシステムの仕様やマウント状態によって書込みが拒否される場合がある．

書込みが成立するには，ファイルシステム自体が書込み操作を提供している必要がある．SquashFS 等の read-only 用途のファイルシステムでは，通常のファイル作成や変更を行うことができない．一方，ext4 や JFFS2 等の書込み可能なファイルシステムであっても，実行時に read-only 状態で利用されている場合には書込みは成立しない．

Linux では，read-only 状態をファイルシステム全体に対応する superblock と，個別の mount の双方に設定できる．したがって，書込み可能なファイルシステムに対して通常の書込みを成立させるためには，superblock および書込み

© 2026 Information Processing Society of Japan

に利用する個々の mount の双方について書込みを妨げる read-only 状態が存在しないことが必要となる．

さらに，IoT 機器では，read-only ファイルシステムと書込み可能なファイルシステムを OverlayFS として統合する構成がみられる．OverlayFS では，lower 側が read-only であっても，書込み可能な upper が存在すれば，merged view に対する変更を upper 側へ保存できる．この場合，書込み成立には，OverlayFS が書込み可能な状態でマウントされていること，upper が存在すること，upper 側のファイルシステムおよび mount が書込み可能であることが条件となる．

3.4 デバイスサブシステムおよびハードウェアによる制約

ファイルシステムより下位では，不揮発性メモリの種類に応じて異なるデバイスサブシステムが書込み要求を処理する．例えば，eMMC 等では block layer および MMC subsystem を経由し，raw flash では MTD subsystem を経由して不揮発性メモリへ書込み要求が送られる．

block layer では，デバイスまたはパーティションにソフトウェア的な read-only 状態を設定できる．この状態は，ファイルシステムを read-write で利用できるかにも影響する．そのため，ファイルシステムが read-write でマウントされている場合には，マウント時点で下位デバイスに関する一部の read-only 条件を通過していると考えられる．

MTD subsystem では，MTD デバイスまたはパーティションの書込み可能状態が管理されており，書込みや消去を行う際にも MTD の書込み可能状態が確認される．このため，上位の VFS やファイルシステムにおいて書込みが許可されている場合でも，MTD 側の書込み保護によって最終的な I/O が拒否される可能性がある．

さらに下位では，SPI Flash や eMMC 等の記憶媒体自身が持つ write protection によって書込みが拒否される場合がある．これらはソフトウェアから確認可能なレジスタによる保護に加え，物理的な WP pin 等，実行中の OS から観測が困難な保護も存在する．本研究では，ツールから取得可能なソフトウェア上の状態を中心に判定し，直接取得できないハードウェアレベルの保護は調査対象外とする．

3.5 書込み成立条件の整理

以上の調査より，通常ファイルに対する書込み要求は，図 1 に示す各層を経由して処理される．書込みが成立するには，実際の書込み経路上で適用されるすべての保護機構において，当該操作が許可される必要がある．

表 1 は，書込み経路に適用され得る主な保護機構と，その判定に用いる設定を整理したものである．ただし，表中の各行が常に独立した AND 条件として一律に適用されるわけではなく，対象パスの書込み経路に応じて適用される条件集合が定まり，その組合せによって書込み可否が決定される．例えば，OverlayFS を介する書込みでは，変更先となる upper 側のファイルシステムおよび mount の条件を評価し，さらにファイルシステムより下位の構成に応じて，block layer や MTD subsystem など実際に経由する層の条件が適用される．このため，調査対象機器に実装されていない機構，または対象パスの書込み経路に適用されない機構は，書込み成立条件には含まれない．

さらに，適用される条件集合に含まれる項目間にも依存関係がある．例えば，POSIX ACL は mode bits とともに DAC の判定を構成し，Linux Capability は一部の DAC 判定を迂回し得る．

4. 書込み成立条件の診断手法

本章では，3 章で整理した書込み成立条件に基づき，Linux ベースの IoT 機器を対象として，各層の状態を実機上で取得し，新規ファイルの作成および自動実行設定の改変可否を判定する診断ツールについて述べる．

4.1 設計方針

本ツールは，対象となるファイルパスについて，実行主体の権限，対象ファイルおよびディレクトリの属性，ファイルシステムと mount の状態，OverlayFS の構成，マウント元に対応する下位デバイスの状態を取得し，どの条件によって書込みが許可または阻害されるかを分析する．

本研究では，IoT マルウェアによる通常ファイルへの書込みを想定し，以下の 2 種類の操作を診断対象とする．

- 新規ファイル作成：マルウェア自身や関連データの保存を想定し，O_WRONLY | O_CREAT を指定したファイル作成を対象とする．
- 既存ファイルへの書込み：起動スクリプトや自動実行設定の変更を想定し，既存ファイルを O_WRONLY で開く操作を対象とする．

両者では，VFS において評価される対象が異なる．新規ファイル作成では親ディレクトリに対する書込み・検索権限が重要となる一方，既存ファイルへの書込みでは，対象ファイル自身の書込み権限およびパス探索に必要なディレクトリの検索権限が必要となる．このため，本ツールでは両操作を別のシナリオとして判定する．

4.2 実装

診断ツールは，Linux ベースの対象機器上で単独実行可能な C 言語プログラムとして実装した．IoT 機器では，CPU アーキテクチャに加え，利用可能な共有ライブラリの構成が機器ごとに異なる．そこで，対象アーキテクチャごとに実行形式を生成するとともに，使用する外部ライブラリを静的にリンクする．診断ツールで利用する外部ライブラリは，POSIX ACL 情報の取得に用いる `libacl` と，Linux Capability 情報の取得に用いる `libcap` である．

© 2026 Information Processing Society of Japan

表 1 書込み経路上の各層における書込み成立条件

層名	機能	書込み成立条件	設定
File	inode flags	inode 属性が対象操作を許可	対象・親の immutable/append-only 設定
System Call	Seccomp	必要なシステムコールが許可	適用 filter の規則
VFS	DAC	実行主体が必要なアクセス権を保持	資格情報，対象・経路の owner/group/mode
VFS	POSIX ACL	ACL による実効権限が要求操作を許可	対象・関係ディレクトリの access ACL
VFS	Linux Capability	Capability を含む権限判定が許可	Effective Capabilities
VFS	MAC/LSM	LSM ポリシーが対象操作を許可	LSM の有効状態，セキュリティポリシー
File System	File System Type	書込み操作を提供	ファイルシステム種別
File System	Superblock Read-Only	superblock が read-write	superblock の状態
File System	Per-mount Read-Only	対象 mount が read-write	mount 属性や mount の状態
File System	OverlayFS	upper への変更反映が可能	upper/work/lower 構成，mount の状態
Device Subsystem	Block layer	下位 block device が書込み可能	blockdev の read-only 状態
Device Subsystem	MMC subsystem	host/device の WP が無効	force_ro，host WP，device WP/lock
Device Subsystem	MTD subsystem	MTD device が書込み・消去可能	MTD_WRITEABLE
Hardware	内部レジスタ	書込み対象アドレスが保護範囲外	不揮発性メモリの status register の状態
Hardware	WP pin	物理的な書込み保護が未設定	基板設計・ハードウェア構成

表 2 既存ファイル診断で探索する自動実行関連のパス

方式	探索対象パス	
init/rc スクリプト	`/etc/inittab`,	`/etc/init.d`,
	`/etc/rc*.d`,	`/etc/rc.d`,
	`/etc/rc.local`	
systemd	`/etc/systemd/system`	
cron	`/etc/crontab`,	`/etc/crontabs/*`,
	`/etc/cron.d`,	`/etc/cron.hourly`,
	`/etc/cron.daily`,	`/etc/cron.weekly`,
	`/etc/cron.monthly`,	`/var/spool/cron`,
	`/var/spool/cron/crontabs`	
シェル起動	`/etc/profile`,	`/etc/profile.d`,
	`/etc/bash.bashrc`,	`/etc/bashrc`

新規ファイル作成の診断： 書込み内容が不揮発性メモリに保持され，かつ実効的に書込み可能と判定されたマウントポイントを探索対象とする．各マウントポイント配下のディレクトリを探索し，一時ファイルの作成によって書込み可否を確認する．診断対象となるディレクトリが見つかった場合は，`open()` で O_WRONLY | O_CREAT を指定して書込み用に開き，fd が取得できた場合は `write()` によって書込みを試行する．

既存ファイルの変更の診断： あらかじめプログラム内に定義した自動実行関連のパスを探索し，発見した通常ファイルを診断対象とする．定義した自動実行関連の探索パスを表 2 に示す．診断対象となるファイルが見つかった場合は，`open()` で書込み用に開き，fd が取得できた場合は `write()` によって書込みを試行する．

書込み可否は，`open()` および `write()` の実行結果と各層から収集したセキュリティ属性，マウント構成，およびデバイス状態を用いて判定する．最終的な診断結果は，3.5 節で整理した書込み成立条件に基づき，書込みが成立した場合を `ALLOWED`，成立しなかった場合を `DENIED`，診断処理を実行できない場合または取得情報のみでは判定できない場合を `UNKNOWN` として出力する．

4.3 書込み成立条件の情報取得

本節では，診断ツールが表 1 で示した書込み成立条件に関係する各層の状態をどのように取得するかを説明する．

System Call 層の情報取得： Seccomp の設定状態については，診断プログラム実行時の `open` および `write` システムコールの実行可否で判定する．

VFS 層の情報取得： VFS 層では，ファイルに対するアクセス可否および操作制約を決定する設定状態を把握するため，DAC，POSIX ACL，プロセス Capability，MAC/LSM ラベル，ならびに inode flags を収集する．

DAC については，対象の指定方法に応じて `lstat()`, `stat()`, `fstatat()`, または `fstat()` から得られる `struct stat` を参照し，ファイルの所有者 UID，所有グループ GID，および mode bits を取得する．さらに，mode bits では表現できないユーザおよびグループ単位の権限を補足する POSIX ACL の設定を `acl_get_file()` により取得し，`acl_get_entry()` を用いて各 ACL エントリを解析する．

Linux Capability については，書込み処理の実行時に `cap_get_proc()` を用いて Capability 集合を取得し，`cap_to_text()` により文字列化する．取得結果から，書込みの権限判定に関連する Effective 集合を抽出する．

また，LSM に基づく強制アクセス制御で参照されるセキュリティラベルを取得する．具体的には，`getxattr()` を用いて，SELinux の `security.selinux` および SMACK の `security.SMACK64` を拡張属性から読み出す．

さらに，ファイル操作を制限する属性である inode flags を確認する．対象ファイルの fd に対して，`FS_IOC_GETFLAGS` を指定した `ioctl()` を実行し，取得したフラグのうち `FS_IMMUTABLE_FL` および `FS_APPEND_FL` を検査する．これ

© 2026 Information Processing Society of Japan

により，対象ファイルが変更不能または追記専用として設定されているかを判定する．

File System 層の情報取得： File System 層では，対象パスから見た実効的な RO/RW 状態，ファイルシステム種別，およびマウント構成を取得する．対象パスに対して `statvfs()` および `statfs()` を実行し，返却された `f_flag` の `ST_RDONLY` フラグから，当該パスを含むマウントが read-only であるかを判定する．また，`statfs()` から得られる `f_type` のマジックナンバーに基づいて，ファイルシステム種別を識別する．さらに，`/proc/self/mountinfo` を解析して対象パスに対応するマウントエントリを特定し，マウントオプション，マウント元，major:minor 番号，およびスーパーブロックのオプションを取得する．

OverlayFS の場合は，`upperdir` および `lowerdir` の構成を取得する．`upperdir` が設定されている場合は，そのパスについてもマウントおよびファイルシステムの状態を取得する．

Device Subsystem 層の情報取得： Device Subsystem 層では，ファイルシステムより下位のデバイスに設定された read-only 状態を取得する．デバイスとの対応付けには，`/proc/self/mountinfo` から得られる major:minor 番号およびマウント元を用いる．ブロックデバイスについては，`/sys/dev/block/<major>:<minor>/ro` を参照し，ブロック層の read-only フラグを取得する．マウント元が`/dev/mtdblock<n>`である場合には，対応する MTD デバイスの`/sys/class/mtd/mtd<n>/flags` を参照し，`MTD_WRITEABLE` フラグの有無からデバイスの書込み可能性を判定する．これらのエントリを取得できない場合は，書込み可能と推定せず `UNKNOWN` として報告する．

5. 実験

本章では，4 章で示した診断ツールを Linux ベースの IoT 機器へ適用し，3 章で整理した書込み成立条件を実機上で取得・判定できるかを 2 つの評価によって検証する．

(1) **検証環境による動作検証：** ファイルシステムおよび不揮発性メモリの構成を事前に把握している GL-MT300N-V2（以降，MANGO とする）を用いて，診断ツールが取得した情報および書込み判定結果と，実際のファイル操作結果を比較する．これにより診断ツールが対象パスのファイルシステム構成や書込み条件を正しく識別できるかを検証する．

(2) **実機における評価：** 2 機種の実 IoT 機器に診断ツールを適用し，各機種における書込み可能なパスと，改変可能な自動実行設定の有無を調査する．これにより，本ツールの探索範囲において，マルウェア永続化に必要な新規ファイル作成および自動実行設定の改変の両条件が成立するかを評価する．

表 3 MANGO におけるマウントポイント別の書込み可否

path	永続化書込み可否	阻害要因
/	○	-
/overlay	○	-
/tmp	×	揮発性領域
/dev	×	揮発性領域
/rom	×	SquashFS
/proc	×	仮想ファイルシステム
/sys	×	仮想ファイルシステム

5.1 検証環境における動作検証

検証方法： MANGO に SSH で root ユーザとしてログインした状態で診断ツールを実行し，まず実行主体の権限，mount 構成，ファイルシステム種別，OverlayFS の構成，ならびにファイルシステムより下位のデバイスに関する情報を取得する．

次に，新規ファイル作成について，診断ツールが抽出した書込み可能な候補パスを対象としてファイルを作成する．また，自動実行設定の変更について，機器上で利用可能な自動実行機構の設定ファイルを対象として書込みを行う．

書込み後に機器を再起動し，以下を確認する．

- 作成したファイルが再起動後も残存するか
- 変更した設定ファイルが再起動後も残存するか
- 設定した自動実行処理が再起動後に実行されるか

これらの実測結果と，診断ツールによって取得された書込み成立条件を比較する．

診断ツールの結果： MANGO 上で診断ツールを実行した結果を表 3 に示す．MANGO 上では，/および`/overlay` が，永続化先として書込み可能なマウントポイントとして判定された．/は OverlayFS で構成されており，書込み内容は upperdir である`/overlay/upper` に反映される．この upperdir は，MTD ブロックデバイス`/dev/mtdblock6` 上の JFFS2 で構成されるため，診断時の実行権限で書き込まれた内容は，再起動後にも保持され得る．一方，`/tmp` および`/dev` は揮発性領域であり，書込み内容は再起動後に保持されない．また，`/rom` は read-only の SquashFS であり，`/proc` および`/sys` は仮想ファイルシステムであるため，いずれも永続化先として利用できない．なお，表中の × は書込み操作そのものが常に不可能であることではなく，書込み内容を永続化できないことを表す．

表 4 は，診断ツールが出力した永続的かつ書込み可能と分類された領域での新規ファイル作成と，検出された自動実行設定ファイルの書込み権限をまとめたものである．

診断ツールは，マウントポイント 7 件のうち，/および`/overlay` を永続的かつ書込み可能な領域として分類した．これらに属する 68 件の候補ディレクトリでは，新規ファイル作成，データ書込み，`fsync`，および読出し確認がすべて成功した．また，一意な自動実行設定ファイル 60 件について，書込み権限があると判定された．

© 2026 Information Processing Society of Japan

表 4　MANGO における永続化に必要な操作の診断結果

操作	対象	書込み可否	永続化への影響
新規ファイル作成	/および/overlay 配下の 68 ディレクトリ	○	不揮発性メモリ上に実行ファイルを保存可能
自動実行設定の改変	/etc/crontabs/root	○	定期実行される cron ジョブ設定を改変可能
自動実行設定の改変	/etc/inittab	○	起動時に参照される設定を改変可能
自動実行設定の改変	/etc/init.d/*（56 ファイル）	○	起動時に実行されるスクリプトを改変可能
自動実行設定の改変	/etc/rc.local	○	起動時の自動実行設定を改変可能
自動実行設定の改変	/etc/profile	○	シェル起動時の自動実行設定を改変可能

永続化の検証結果： 診断ツールにおいて，OverlayFS の upper には JFFS2 が利用されていることを確認した．また，JFFS2 のマウント元は /dev/mtdblock6 である．永続化の確認として，/root/hello.sh を作成し，cron の設定に同スクリプトを定期的に実行する処理を追加した．これらの変更はそれぞれ OverlayFS の upper 側へ保存された．機器を再起動した後もスクリプトおよび cron 設定が残存し，設定した処理が実行されることを確認した．このことから，当該構成では OverlayFS の merged view を介した書込みが upper 側の不揮発領域へ反映され，再起動後にも利用されることが確認できた．

5.2　実機を用いた評価

MANGO を用いた検証において，診断ツールを利用することで永続化するために必要な不揮発性メモリが関係するマウントポイントの情報とファイルの新規作成および自動実行設定の改変の両条件が成立したときにマルウェアが永続化可能となることを示した．そこで，国内市場で購入可能な SOHO ルータの実機 2 機種（製品 A，B）に対して診断ツールを実行することでファイルシステムを介したマルウェア永続化が可能であるのかを調査した．

評価方法： 調査対象機器において，書込み内容が不揮発性メモリに保持されるファイルシステム上に，ファイルを新規作成できるパスや改変可能な自動実行設定ファイルが存在するかを調査し，永続化の成立可能性を評価する．本評価では，購入時点のファームウェアバージョンを使用し，設定画面からリモートアクセス機能を有効にし，ツールを実行した．なお，いずれの機種もリモートアクセス機能のログインユーザは root であった．

評価結果： 表 5 に評価結果を示す．製品 A では，JFFS2 上の/jffs に新規ファイルを作成できたことから，マルウェアを保存可能な領域が存在した．一方で，自動実行設定として検出した/etc/profile は SquashFS 上にあり，改変できなかった．これは，3 章で整理したファイルシステムの種類が自動実行設定への書込みを実際に阻害した例である．したがって，本ツールの探索範囲では，マルウェアの保存は可能であるが，再起動後の自動実行に必要な条件を満たさなかった．

製品 B では，JFFS2 を upper とする OverlayFS 上に新規ファイルを作成でき，検出した 59 件の自動実行設定もすべて改変可能であった．本実験において，root 権限下では，これらの操作に対してアクセス制御や書込み保護が阻害要因となる事例は確認されず，通常のファイル操作のみで，マルウェアの保存と自動実行設定の改変の双方が成立した．したがって，本ツールの探索範囲では，製品 B は本研究で対象とする永続化の成立条件を満たした．

表 5　SOHO ルータ 2 機種の評価結果

製品	新規ファイル作成	自動実行設定の改変	永続化可能性
A	○(4 箇所)	×(1 箇所)	×
B	○(68 箇所)	○(59 箇所)	○

5.3　書込み成立条件に基づく評価結果の考察

MANGO および実機 2 機種の評価結果を表 1 の各層と対応付けると，本評価において書込みを阻害する要因として確認されたのは，File System 層のファイルシステム種別であった．一方，本ツールの診断対象としたその他の層では，本実験の root 権限下で対象としたパスおよび操作に対して，書込みを阻害する要因は確認されなかった．3 章では，Linux における各層の書込み成立条件を整理したが，本評価の範囲では，これらの条件によって実際に書込みが阻害された事例は限定的であった．

特に製品 B では，ファイルの新規作成および自動実行設定の改変のいずれに対しても，書込みを阻害する要因は確認されなかった．この結果から，Linux には書込みを制限し得る複数の機構が存在していても，IoT 機器の構成によっては，それらが永続化に必要な書込みに対する実効的な制約として機能しない場合があると考えられる．このような構成では，root 権限でマルウェアが実行可能となった場合，書込み保護の解除や迂回といった追加の操作を行わずとも，通常のファイル操作と機器に備わる自動実行機能を利用して，本研究で対象とする永続化を達成し得る．

6.　議論

本章では，3 章で整理した不揮発性メモリへの書込み成立条件と，5 章の実機評価結果に基づき，永続化攻撃への対策について考察し，本研究の制約を示す．

© 2026 Information Processing Society of Japan

6.1 書込み成立条件を踏まえた永続化対策

5.3 節で示したように，実機評価では，書込みを阻害する要因として確認されたものは限定的であった．この背景の一つとして，IoT 機器における保護機構の利用状況が考えられる．VFS 層の MAC/LSM について，三宅ら [10] は，IoT 機器のファームウェアを対象とした大規模調査により，近年の IoT 機器においても LSM ベースの MAC が十分に利用されていないことを報告している．こうした調査結果を踏まえると，本評価において MAC/LSM が書込みを阻害する要因として現れなかった背景の一つとして，IoT 機器における MAC/LSM の導入が限定的であることが考えられる．

これに対し，本評価では，File System 層のファイルシステム種別が実際に書込みを阻害する要因となった．この結果を踏まえると，通常運用時に変更を必要としない自動実行設定については，SquashFS 等の書込み操作を提供しないファイルシステム上に配置することで，マルウェアによる改変を阻害できると考えられる．一方，設定保存等のために書込みを許可する必要がある場合には，書込み操作を提供しないファイルシステム上に配置する方法は適用できない．このような場合には，VFS 層の MAC/LSM 等，異なる層の保護機構を利用し，実行主体や対象パスに応じて書込みを制限することが考えられる．したがって，永続化対策では，各層に保護機構が存在することだけでなく，マルウェアの保存や自動実行設定の改変に必要な書込みに対して，書込み経路上のいずれかの機構が実際に書込みを阻害する構成とすることが重要である．

ただし，書込みの制限は，ファームウェア更新や設定保存等の正規処理にも影響し得るため，対策の設計では保守に必要な書込みとの両立を考慮する必要がある．

6.2 制約

実機評価では，リモートアクセス機能を用いて root ユーザとしてログイン可能な機種のみを対象とした．実際の攻撃では，Web サーバ等の脆弱性を起点としてコードが実行される場合があり，その実行主体の UID，Linux Capability，LSM，Seccomp 等の制約は本実験と異なり得る．したがって，本結果は主として root 権限取得後における書込みおよび永続化の成立可能性を示すものであり，初期侵入や権限昇格を含む攻撃全体の成立を示すものではない．

また，診断ツールの動作検証に用いた機器は 1 機種，実機評価の対象は国内市場で入手可能な SOHO ルータ 2 機種に限られる．IoT 機器のファイルシステム構成や書込み保護機構は，機器種別，ベンダ，ファームウェアバージョンおよび設定によって異なる．したがって，Linux ベースの IoT 機器全体において永続化が成立し得る構成を備えた機器の割合を推定するには，これらの差異を考慮して対象機種を拡大した評価が必要である．

7. おわりに

本研究では，Linux ベースの IoT 機器におけるマルウェア永続化を対象として，ユーザ空間から不揮発性メモリに至る書込み経路について，各層で書込みの成否を決定する要因を整理し，それらの組合せに基づく書込み成立条件を示した．また，これらの条件に関する情報を実機から取得するツールを開発し，MANGO および 2 機種の SOHO ルータを用いて書込み可否を評価した．その結果，MANGO では診断結果と実際の動作が一致することを確認し，SOHO ルータでは，両機種とも作成したファイルが再起動後も保持された一方，自動実行設定の改変可否は機種間で異なった．これらの結果を踏まえ，実機において書込みを阻害する要因が限定的であった背景を検討するとともに，永続化に必要な書込みに対して，書込み経路上のいずれかの保護機構が実際に書込みを阻害する構成とすることの重要性について考察した．今後は，ツールの適用対象を拡大するとともに，実マルウェアによる永続化の成否と診断結果を比較し，IoT 機器におけるマルウェア永続化の脅威を分析する．

参考文献

[1] Antonakakis, M. et al.: Understanding the Mirai Botnet, *26th USENIX security symposium (USENIX Security 17)*, pp. 1093–1110 (2017).

[2] Griffioen, H. et al.: Examining Mirai's Battle over the Internet of Things, *The 2020 ACM SIGSAC Conference on Computer and Communications Security*, pp. 743–756 (2020).

[3] Brierley, C. et al.: Persistence in Linux-Based IoT Malware, *Secure IT Systems: 25th Nordic Conference*, pp. 3–19 (2020).

[4] Alrawi, O. et al.: The Circle Of Life: A Large-Scale Study of The IoT Malware Lifecycle, *The 30th USENIX Security Symposium*, pp. 3505–3522 (2021).

[5] 原悟史ほか：感染持続型 IoT マルウェアの実態調査と実機による概念実証, 電子情報通信学会論文誌 B，Vol. J102-B, No. 8, pp. 524–535 (2019).

[6] Rodríguez, E. et al.: Difficult for thee, but not for me: Measuring the difficulty and user experience of remediating persistent IoT malware, *2022 IEEE 7th European Symposium on Security and Privacy (EuroS&P)*, IEEE, pp. 392–409 (2022).

[7] Largent, W.: New VPNFilter malware targets at least 500K networking devices worldwide, `https://blog.talosintelligence.com/vpnfilter`.

[8] Hau, B. et al.: SYNful Knock - A Cisco router implant - Part I, `https://cloud.google.com/blog/topics/threat-intelligence/synful-knock-acis`.

[9] 添田隼喜ほか：IoT 機器に対するマルウェア持続感染性の診断手法の提案, 信学技報, vol. 122, no. 442, pp. 181–186 (2023).

[10] 三宅貴義ほか：ファームウェア解析に基づく IoT 機器のセキュリティ機能に関する大規模実態調査〜Linux ベースセキュリティ機能の導入実態と複数機能の組合せ分析〜, 信学技報, vol. 126, no. 111，pp. 175–182 (2026).

© 2026 Information Processing Society of Japan